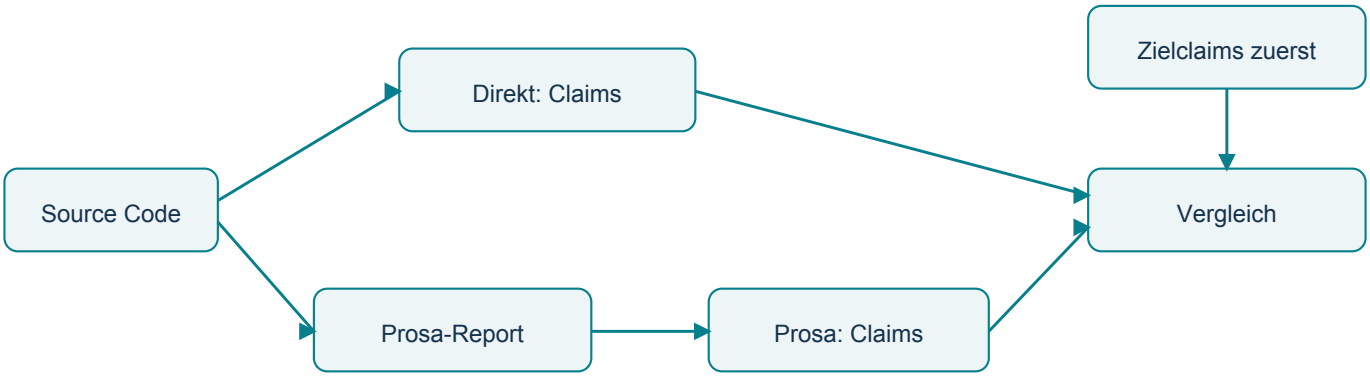
What Can We Claim?

Generating Security Claims from Code and Prose with LLMs

Drei Entwicklungsfälle - direkter Weg und Prosa-Zwischenschritt

Warum einzelne Claims?

Ein ganzer Report kann richtige Codebeobachtungen und unbelegte Angriffswirkungen vermischen. Einzelne, eigenständig verständliche Aussagen erlauben gezielte Teilurteile: Was ist belegt, was fehlt und welche Bedingung wurde verändert? Kontext und Evidenz bleiben sichtbar [1-3]. IDs und Git machen Änderungen nachvollziehbar; Claims sind nicht von sich aus versioniert.



Beide Wege verwenden dieselben Codebytes. Der Vergleich greift auf vorher fixierte Zielclaims zu; die Generierung erhält diese Referenz nicht. Untersucht werden Inhalte und Erzeugungswege, nicht nur eine Formatumwandlung [4].

Drei Beispiele aus Vul4J [5]

Fall	Bereitgestellter Code / Grenze
JSPWiki VUL4J-18	Fünf vollständige Dateien; WikiEngine, Filter und Ziel-JSPs fehlen.
Jackson XML VUL4J-47	XmlFactory.java, Originalzeilen 84-144; Provider und Aufrufer fehlen.
Commons YAML VUL4J-9	YAMLConfiguration.java vollständig; SnakeYAML und Aufrufer fehlen.

Was sind die Basis- oder Zielclaims?

Codegestützte Sollinhalte nach dem Muster der früheren SG-Annotation: etwa JSPWiki-Pfad versus Query, Jacksons bedingte Factory-Konfiguration und YAML-Ladepfade mit bzw. ohne Optionen. Das gemeinsame JSON enthält ID, Proposition, Familie, Begründung, Bedingungen/Scope, Quellenzitate mit Zeilen und Kontextbezüge. Bedingungen stehen auch im Satz; keine feste Claimzahl.

Wie wurde geprüft?

Zielreferenz und semantischer Vergleich sind KI-unterstützte Entwürfe von Codex; menschliches Gegenlesen steht aus. Die historischen SG-Claims sind keine unabhängige Codereferenz. Automatisch werden Format, Zitate und Referenzen geprüft. Formal abgelehnte Rohclaims bleiben abgelehnt; ihre Propositionen werden nur diagnostisch gegengelesen. Inhaltliche Zuordnungen sind einzeln begründet, ohne automatisches Matching.

Modell: gpt-5.4-mini-2026-03-17; ein Lauf je Fall und Weg, insgesamt neun API-Aufrufe ohne Retry. Geschätzte API-Kosten: 0,091 USD bei 1 USD Kostenrahmen. Keine neuen Java-/PoV-Läufe.

Ergebnisse im direkten Vergleich

Deskriptive Zusammenfassung der eingetragenen KI-Bewertung; keine menschlichen Messurteile.

Fall	Basis	Direkt: Rohclaims / Form	Prosa: Claims / Form	Abdeckung D V/T/F	Abdeckung P V/T/F
JSPWiki	9	9 / abgelehnt	10 / gültig	3/4/2	0/4/5
Jackson XML	6	5 / abgelehnt	12 / gültig	3/1/2	2/1/3
Commons YAML	5	6 / abgelehnt	8 / gültig	3/1/1	0/3/2

V/T/F = vollständig / teilweise / fehlend abgedeckte Zielinhalte. Bei abgelehnter Direkt-Ausgabe ist diese Zuordnung nur eine Rohclaim-Diagnose. Prosa-Zitate verweisen auf den Report; Reporttreue und Codeunterstützung werden gesondert beurteilt. Claimzahl ist kein Qualitätsscore; mehrere Claims können gemeinsam einen Inhalt abdecken.

JSPWiki: konkretes Beispiel

Basis B07: WikiServlet.doGet setzt m_engine.encodeName(pageName) nach dem Literal ?page= in das Dispatcherargument ein und hängt anschließend & sowie req.getQueryString() an; pageName bildet damit den behandelten Querywert und nicht die Variable jspPage des Pfadanteils.

Direkt-Rohclaim C2: WikiServlet.doGet(...) baut den Forward-Zielpfad aus m_engine.getURLConstructor().getForwardPage(req) und m_engine.encodeName(pageName) zusammen und hängt zusätzlich "&" + req.getQueryString() an die an RequestDispatcher übergebene Forward-URI an.

Prosa C3: Die Datenflüsse sind damit als Request-Pfad > Seitennamenextraktion > URL-Encoding > serverseitige Weiterleitung beschrieben.

Befund: Direkt bleiben mehr Parserdetails erhalten, doch Pfad und Query werden vermischt. Im Report fehlen weitere Zielinhalte; die treue Extraktion übernimmt außerdem die unbelegte Aussage „unvalidiert“. Beide Wege bleiben fachlich unvollständig.

Jackson XML: konkretes Beispiel

Basis B01: Wenn xmlIn null ist, erzeugt der gezeigte XmlFactory-Hauptkonstruktor eine XMLInputFactory über newInstance() und ruft auf ihr setProperty(IS_SUPPORTING_EXTERNAL_ENTITIES, Boolean.FALSE) auf.

Direkt-Rohclaim C1: Wenn der Konstruktor XmlFactory(ObjectCodec oc, XMLInputFactory xmlIn, XMLOutputFactory xmlOut) mit xmlIn == null aufgerufen wird, erzeugt der Code eine neue XMLInputFactory per XMLInputFactory.newInstance() und setzt auf dieser anschließend XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES auf Boolean.FALSE.

Prosa C4: Durch diese Konfiguration werden externe Entitäten standardmäßig deaktiviert.

Befund: Direkt wird die Nicht-null-Asymmetrie genauer erfasst. Beide Wege überdehnen teils den Setter zur Schutzwirkung. DTD-Setter-Abwesenheit und Kopierkonstruktor fehlen beiden; zusätzliche Prosa-Claims ersetzen diese Inhalte nicht.

Commons YAML: konkretes Beispiel

Basis B02: Die gezeigten read(Reader, LoaderOptions)- und read(InputStream, LoaderOptions)-Overloads erzeugen jeweils new Yaml(options) und übergeben ihre Eingabe in an yaml.load(in).

Direkt-Rohclaim C2: Die beiden read(..., LoaderOptions)-Überladungen verwenden die vom Aufrufer übergebenen LoaderOptions beim Erzeugen des Yaml-Objekts, während die Parameterlos-Varianten new Yaml() ohne übergebene Optionen verwenden.

Prosa C5: Für das Schreiben wird explizit ein Block-Format gewählt, und es existieren Varianten der read-Methoden, die LoaderOptions an SnakeYAML weiterreichen.

Befund: Direkt bleiben Optionsvarianten und Catch/rethrow konkreter. Die Prosa verliert diese Details und die lokale Tag-/Typprüfungsnegation. Korrekte zusätzliche Schreibpfadclaims sind kein Ersatz für die fehlenden Ladeinhalte.

Antworten und Reichweite

RQ0 - Warum Claims?

Reports können korrekte Codebeobachtungen und unbelegte Wirkungen vermischen. Explizite Claims machen Teilurteile, Voraussetzungen und Quellen adressierbar [1-3]. IDs und Git schaffen Nachvollziehbarkeit. Das ist eine literaturgestützte Motivation; ein Zeit- oder Sicherheitsgewinn wird hier nicht gemessen.

RQ1 - Wie werden Claims erzeugt?

Ein LLM-Durchgang erzeugt Claims im gemeinsamen JSON-Schema. Der Prompt verlangt beurteilbare Propositionen, erhaltene Bedingungen und exakte Quellen. Direkt wird nur Code gelesen; die Prosa-Extraktion liest ausschließlich den neuen Report. Danach prüft lokaler Code Form und Zitate, keine Wahrheit.

RQ2 - Wie brauchbar sind sie gegenüber den Zielclaims?

Die einzeln begründete KI-Zuordnung zeigt in beiden Wegen Auslassungen, Bündelungen und zu starke Schutz-/Wirkungsaussagen. Manche Zusatzclaims sind korrekt. Gegenüber einer unabhängig menschlich bestätigten Referenz ist die Qualität noch nicht beantwortet: Auch die neuen Basisclaims sind KI-unterstützte Entwürfe.

RQ3 - Direkt oder über Prosa: was ist besser?

Hier erhalten direkte Rohclaims mehr konkrete Zielinhalte; alle drei Direktantworten verletzen aber den exakten Zitiervertrag. Alle drei Prosaextraktionen sind formal gültig, übernehmen jedoch Reportauslassungen und Überstärkungen. Ein pauschaler Sieger folgt daraus nicht; Formatzuverlässigkeit und Inhaltsabdeckung zeigen unterschiedliche Stärken.

Gesamtfazit

Der MVP macht den Unterschied zwischen Quellenfehler, Reportverlust und unvollständiger Zerlegung konkret sichtbar. Die Prosa ist in diesem Lauf formal zuverlässiger, die direkte Rohantwort inhaltlich näher an den gewählten Zielinhalten. Keiner der Wege liefert durchgehend vollständige, unverändert nutzbare Claims.

Grenzen und nächster Schritt

- Drei bekannte Entwicklungsfälle, ein Lauf je Weg; keine unabhängige Stichprobe oder statistische Überlegenheit.
- Zielreferenz und Vergleich sind KI-unterstützt, menschliche Kontrolle steht aus; keine Humanstudie oder Nutzenmessung.
- Begrenzte Codepakete, keine neuen Builds oder PoV-Läufe; Datasetlabel und Claimlinks beweisen keinen Exploit.

Nächster Schritt: Zielclaims und Zuordnungen mit SG fachlich gegenlesen. Danach Zitierprompt und Granularität gezielt überarbeiten und einen neuen Versuch gesondert festlegen.

Primärquellen

[1] Miriam Wanner et al. (2024). A Closer Look at Claim Decomposition

[2] Yixiao Song et al. (2024). VeriScore: Evaluating the factuality of verifiable claims in long-form text generation

[3] Object Management Group (2022). Structured Assurance Case Metamodel (SACM), Version 2.2

[4] Zhi Rui Tam et al. (2024). Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models

[5] Vul4J: fixierte Datasetfassung und drei ausgewählte Fälle.